

Le Quốc Việt CNTT 5

1350080350

B1. ~~interface~~

```
interface Mail {  
    void method sendMail(); log  
    void method log();  
}
```

```
class MailImpl implements Mail {
```

```
    @Override
```

```
    public void sendMail () {
```

```
        System.out.println ("Đã gửi mail");
```

```
    }  
    @Override
```

```
    public void log () {
```

```
        System.out.println ("Đang log");
```

```
    }  
}
```

```
class main {
```

```
    public void static static void main (String [] args) {
```

```
        MailImpl m1 = new MailImpl();
```

```
        m1.sendMail();
```

```
        m1.log();
```

```
    }  
}
```

B1:

```
interface Shape { → String color = "red";  
    void draw();  
}  
  
public class Rectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("new." + color + " rectagle");  
    }  
}  
  
public class Circle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("new." + color + " circle");  
    }  
}  
  
class Main {  
    public static void main (String[] args) {  
        Shape rect = new Rectangle();  
        Shape circ = new Circle();  
        rect.draw();  
        circ.draw();  
    }  
}
```

B4

```
interface Car {  
    public String color;  
    public double speed;  
    void Accelerate();  
    void  
    public String get Color, {  
        return red;  
    }  
    public double getSpeed {  
        return 100;  
    }  
}  
  
class Bmw extends implements Car {  
    @Override  
    void Accelerate {  
        System.out.println("Xe Bmw đang ");  
    }  
    @Override  
    public String toString {  
        return "Màu xe " + super.getColor + " Tốc độ: " +  
        super.getSpeed;  
    }  
}
```

```
class Civic implements Car {
```

```
    @Override
```

```
    void Accelerate ()
```

```
    System.out.println("Civic say");
```

```
}
```

```
    @Override
```

```
    public String toString ()
```

```
    return "Civic" + super.getColor() + " for do" + super
```

```
getSpeed();
```

```
}
```

```
}
```

```
class main {
```

```
    public static void main (String [] args) {
```

```
        BMW myBMW = new BMW ();
```

```
        Civic myCivic = new Civic ();
```

```
        myBMW.Accelerate ();
```

```
        myCivic.Accelerate ();
```

```
}
```

```
}
```

135.

```
interface Movable {  
    void moveUp ();  
    void moveDown ();  
    void moveLeft ();  
    void moveRight ();  
}
```

```
class MovablePoint implements Movable {  
    public int x ;  
    public int y ;  
    public int xSpeed ;  
    public int ySpeed ;  

```

```
    public MovablePoint ( int x, int y, int xSpeed ,  
        int ySpeed ) {
```

```
        this.x = x ;
```

```
        this.y = y
```

```
        this.xSpeed = xSpeed ;
```

```
        this.ySpeed = ySpeed ;
```

```
    }
```

@Override

```

public void moveUp () {
    System.out.println ("Di len");
}

```

```

4 @Override
public void moveDown () {
    System.out.println ("Di xuống");
}

```

```

3 @Override
public void moveLeft () {
    System.out.println ("Sang trái");
}

```

```

1 public void moveRight () {
    System.out.println ("Sang phải");
}

```

```

}
    point extends
    class Movable extends Movable Movable {

```

```

    public int radius;

```

```

    public Movable point center;

```

```

    public void Movable (Movable point, int radius) {

```

```

        this.center = Movable point;

```

```

        this.radius = radius;
    }
}

```

```
public String toString () {  
    return "vtri " + abn + center + " " + rad + radius + "  
}
```

```
public String toString ()
```

```
@Override
```

```
public void MoveUp () {
```

```
    System.out.println (" Circle Up ");
```

```
@Override
```

```
public void MoveDown () {
```

```
    System.out.println (" Circle Down ");
```

```
@Override
```

```
public void MoveLeft () {
```

```
    System.out.println (" Circle Left ");
```

```
public void MoveRight () {
```

```
    System.out.println (" Circle Right ");
```

```
class main {
```

```
    public static void main (String [] args) {
```

```

MovablePoint m1 = new MovablePoint ();
m1.MovablePoint (1, 2, 3, 4);
System.out.println (m1.toString ());
m1.moveUp ();
m1.moveDown ();
m1.moveLeft ();
m1.moveRight ();
MovableCircle c1 = new MovableCircle (m1, 5)
System.out.println ("c1.toString (");
c1.moveUp ();
c1.moveDown ();
c1.moveLeft ();
c1.moveRight ();

```

B6:

```

interface IEmployee {
    int calculateSal ();
    String getFame ();
}

```

```

class employee implements IEmployee {
    private String name;
}

```

GIBOOK

publi

```

    in CalculateSalary () {
        return this.WorkingHours * 22.00;
    }
}

class FullTime extends Employee () {
    public void FullTimeEmp () {
        return "te" + super.name + " length : " + super
        program + perform;
    }
}

```

```

Override
in CalculateSal () {
    return 10 * super.program + perform;
}

```

```

}

class main {
    public static void main (String [] args) {
        partition p1 = new partition ();
        FullTime f1 = new FullTime ();
        p1
    }
}

```